KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

HOSTEL ROOM ALLOCATION AND STUDENT STAY MANAGEMENT SYSTEM

Submitted by

Neharihaa R

YOUR REGISTER NO

25BIT034

Under the Guidance of

DHILLEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled “Hostel Room Allocation and Student Stay Management**”** submitted in partial fulfillment of the requirements for the course [**Course Name]** is a **record of my original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(Neharihaa R)

CERTIFICATE

This is to certify that the project entitled **“PROJECT TITLE”** is a **bona fide work** carried out by **[Neharihaa] [B.Sc.IT-1-A/2025]**, [B.Sc.Information Technology**]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[[Hostel Room Allocation and Student Stay Management System]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: coimbatore

Date:09.04.2026

Submitted for the Viva-Voce Examination held on

____09.04.2026_____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	

8	**References**	

ABSTRACT

The Hostel Room Allocation and Student Stay Management System is a Java-based application developed to simplify hostel room management and student accommodation. The main purpose of this project is to reduce manual work and improve the accuracy of hostel records. The system provides features such as student registration, room allocation, room availability checking, student details display, and check-in and check-out management. It uses Java programming concepts such as classes, objects, methods, loops, and collections to perform various operations. The system checks room availability and capacity before allocating a room to a student. This project helps hostel administrators manage student accommodation efficiently, save time, and reduce errors. In the future, the system can be enhanced with database connectivity, fee management, and a graphical user interface.

1. INTRODUCTION

The Hostel Room Allocation and Student Stay Management System is a Java-based project designed to manage hostel rooms and student accommodation efficiently. In many colleges, hostel room allocation and student details are maintained manually, which can lead to errors and confusion. This system helps hostel administrators register students, allocate rooms, check room availability, and manage student stay details. It reduces paperwork, saves time, and improves the accuracy of hostel management. 1.1 Objective • To maintain student information digitally. • To allocate hostel rooms to students. • To check room availability easily. • To prevent allocating rooms beyond their capacity. • To manage student check-in and check-out details. • To reduce manual work and paperwork. • To provide accurate and organized hostel records.

1.2 Overview

The application helps hostel administrators maintain student details, allocate rooms, check room availability, and manage student check-in and check-out records. Students are registered with an ID and name, and rooms are allocated based on availability and capacity. The administrator can view student records and check occupied and vacant beds. When a student checks out, the room becomes available for another student.

1.3 FEATURES • Student registration and details management. • Hostel room allocation based on room availability. • Room vacancy and capacity checking. • Displaying allocated rooms and student records. • Student check-in and check-out management. • Prevention of duplicate student IDs. • Prevention of room allocation beyond capacity. • Simple menu-driven interface. • Clear messages for invalid choices and unavailable rooms. • Possibility of future database integration. Benefits The system reduces the time spent on manual records, makes room vacancy information easier to access, and organizes student information in one place. Its menu-driven structure is suitable for a basic academic Java project and can be expanded with additional hostel services.

2. PROGRAM ANALYSIS

The system is developed using Java programming to simplify hostel administration. It manages student registration, room allocation, room availability, student records, and check-out operations. Classes and objects represent student details, methods perform specific tasks, conditional statements validate choices, and loops display the menu repeatedly until the user exits. An ArrayList stores student records, while an array represents room capacity. The system checks whether a student is registered before allocating a room and verifies room availability to prevent over-allocation. It also displays student details and occupied and vacant beds. The program can be executed using Visual Studio Code, Eclipse, or IntelliJ IDEA with the Java Development Kit (JDK). In this basic version, records are stored temporarily in memory.

2.1 Problem Definition

Manual hostel management can make it difficult to maintain student details, allocate rooms, identify vacancies, and track check-in and check-out records. These tasks may take time and can result in incorrect records or duplicate allocations. This project provides a computerized solution using Java to register students, allocate rooms based on availability, view student details, and manage room vacancies more efficiently.

2.2 Project Overview

The application presents a menu with options to register a student, allocate a room, view student records, check room availability, check out a student, or exit. It validates student IDs and room capacity before allocation. During check-out, the room is released while the student registration record remains available in the current program session.

2.3 Scopes and Limitations

Scope

• Can be used as a basic room-management tool for college or private hostels. • Maintains student IDs, names, and room allocation status. • Checks the occupancy and vacancy of rooms. • Supports student check-out and room release. • Can be extended with a database, fee

management, attendance, and complaint tracking.

Limitations • Student records are stored in memory and are lost when the program closes. • The basic version uses a console interface rather than a graphical interface. • Online access and multi-user operation are not included. • Fee payment, notifications, and advanced user authentication are not included. • Permanent storage requires file handling or database connectivity. Hardware and Software Requirements Hardware: Computer or laptop, keyboard, monitor, and at least 4 GB RAM. Software: Windows/Linux/macOS, Java Development Kit (JDK), and an IDE such as Visual Studio Code, Eclipse, or IntelliJ IDEA

3. SYSTEM DESIGN

System design describes the structure of the application and the way its components work together. The administrator selects an operation from the menu, the Java program processes the request, and the relevant student or room information is displayed.

3.1 Modules

1. Student Registration Module Accepts a unique student ID and student name. Duplicate IDs are rejected to avoid repeated records. 2. Room Allocation Module Checks that the student exists and does not already have a room. It searches rooms and allocates the first room with available capacity. 3. Student Display Module Displays registered students with their IDs, names, and room numbers. Students without an allocated room are shown as not allocated. 4. Room Availability Module Counts students assigned to each room and displays occupied and vacant beds. 5. Check-out and Exit Modules Check-out releases a student's room while retaining the registration record. The exit option ends the program.

4. METHODOLOGY

The project follows a structured development process. The problem is identified first, requirements are collected, and the system design is prepared before coding. The Java program is then developed, tested with different inputs, and documented with sample outputs.

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the needs of restaurant operations.

The main problems identified in manual billing systems are:

The main problems in manual hostel administration are time-consuming record maintenance, difficulty finding vacant rooms, possible duplicate allocation, and mistakes in student information. The proposed system addresses these issues with registration validation, capacity checking, and organized display of records.

4.2. System Design

The program is divided into small methods so that each operation has a clear responsibility. The Student class stores the ID, name, and allocated room number. An ArrayList stores student objects, and an integer array stores the capacity for rooms 101 to 105. The main method displays a menu and calls the selected method.

4.3. Program Development

The application is implemented in Java. A Student class represents each student, and the HostelManagement class contains the main menu and operations. The program uses an ArrayList to keep student records during execution. The room allocation method counts the students already assigned to each room and assigns a room only when its capacity has not been reached.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case	**Expected Result**
Select menu item	Item displayed correctly
Enter quantity	Correct quantity accepted
Calculate bill	Correct total displayed
Invalid input	Error handled properly

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system generates the final bill based on the selected food items and quantities.

The results are demonstrated through program execution and screenshots showing the input and output of the system.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

Algorithm for the **Hotel & Restaurant Billing System**:

1. Start the program.
2. Display the list of food items with prices.
3. Ask the user to select a food item.
4. Enter the quantity of the selected item.

5. Calculate the total cost of the item.
6. Add the cost to the total bill.
7. Repeat steps 3–6 for additional items if required.
8. Calculate the GST or tax amount.
9. Calculate the final bill amount.
10. Display the final bill to the user.
11. End the program.

4.7. Flowchart

Students should draw a flowchart showing program flow.

Example flow:

Start
↓
Display Menu
↓
Select Item
↓
Enter Quantity
↓
Calculate Bill
↓
Display Invoice
↓
End

Students can draw using:

- draw.io
- Lucidchart
- MS Visio

5. SOURCE CODE

Save the following program as HostelManagement.java. It demonstrates student registration, room allocation, student display, room availability, check-out, and exit operations.

```
import java.util.Scanner;

class Student {
    int id;
    String name;
    String department;
    int roomNumber;

    Student(int id, String name, String department, int roomNumber) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.roomNumber = roomNumber;
    }

    void displayStudent() {
        System.out.println(
            id + "\t" + name + "\t" + department + "\tRoom " + roomNumber
        );
    }
}
```

```
class Hostel {
    private Student[] students;
    private int capacity;
    private int count;

    Hostel(int capacity) {
        this.capacity = capacity;
        students = new Student[capacity];
        count = 0;
    }

    // Allocate a room to a student
    void allocateRoom(int id, String name, String department) {
        if (count >= capacity) {
            System.out.println("Sorry! No rooms available.");
            return;
        }

        if (findStudent(id) != null) {
            System.out.println("Student ID already exists.");
            return;
        }

        int roomNumber = count + 101;

        Student student = new Student(
            id, name, department, roomNumber
        );
```

```
    students[count] = student;
    count++;

    System.out.println("Room allocated successfully!");
    System.out.println("Student Name: " + name);
    System.out.println("Room Number: " + roomNumber);
}

// Find a student using student ID
private Student findStudent(int id) {
    for (int i = 0; i < count; i++) {
        if (students[i].id == id) {
            return students[i];
        }
    }
    return null;
}

// Vacate a student's room
void vacateRoom(int id) {
    int index = -1;

    for (int i = 0; i < count; i++) {
        if (students[i].id == id) {
            index = i;
            break;
        }
    }
```

```
    if (index == -1) {
        System.out.println("Student not found.");
        return;
    }

    System.out.println(
        "Room " + students[index].roomNumber +
        " vacated by " + students[index].name
    );

    // Remove the student and shift remaining records
    for (int i = index; i < count - 1; i++) {
        students[i] = students[i + 1];
    }

    students[count - 1] = null;
    count--;

    System.out.println("Room vacated successfully!");
}

// Display all students staying in the hostel
void viewStudents() {
    if (count == 0) {
        System.out.println("No students in the hostel.");
        return;
    }

    System.out.println("\n--- HOSTEL STUDENT DETAILS ---");
```

```
        System.out.println("ID\tName\tDepartment\tRoom No.");

        for (int i = 0; i < count; i++) {
            students[i].displayStudent();
        }
    }

    // Display available rooms
    void availableRooms() {
        System.out.println("Total Rooms     : " + capacity);
        System.out.println("Occupied Rooms  : " + count);
        System.out.println("Available Rooms : " + (capacity - count));
    }
}

public class HostelManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Hostel hostel = new Hostel(5);
        int choice;

        do {
            System.out.println("\n================================");
            System.out.println(" HOSTEL ROOM ALLOCATION SYSTEM");
            System.out.println("================================");
            System.out.println("1. Allocate Room");
            System.out.println("2. Vacate Room");
            System.out.println("3. View Students");
```

```
System.out.println("4. View Room Availability");
System.out.println("5. Exit");
System.out.print("Enter your choice: ");

while (!sc.hasNextInt()) {
    System.out.println("Please enter a valid number.");
    sc.next();
    System.out.print("Enter your choice: ");
}

choice = sc.nextInt();
sc.nextLine();

switch (choice) {
    case 1:
        System.out.print("Enter Student ID: ");
        while (!sc.hasNextInt()) {
            System.out.println("Enter a valid ID.");
            sc.next();
            System.out.print("Enter Student ID: ");
        }
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Department: ");
        String department = sc.nextLine();
```

```
            hostel.allocateRoom(id, name, department);
            break;

        case 2:
            System.out.print("Enter Student ID to vacate: ");
            while (!sc.hasNextInt()) {
                System.out.println("Enter a valid ID.");
                sc.next();
                System.out.print("Enter Student ID to vacate: ");
            }
            int vacateId = sc.nextInt();
            sc.nextLine();

            hostel.vacateRoom(vacateId);
            break;

        case 3:
            hostel.viewStudents();
            break;

        case 4:
            hostel.availableRooms();
            break;

        case 5:
            System.out.println("Thank you for using the system!");
            break;
```

```
                default:
                    System.out.println("Invalid choice. Try again.");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

6. INPUT AND OUTPUT SCREENS

==========

HOSTEL ROOM ALLOCATION SYSTEM

==========

1. Allocate Room

2. Vacate Room

3. View Students

4. View Room Availability

5. Exit

Enter your choice: 1

Enter Student ID: 101

Enter Student Name: Arun

Enter Department: BCA

Room allocated successfully!

Student Name: Arun

Room Number: 101t

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Hostel Room Allocation and Student Stay Management System demonstrates how Java programming can be used to solve a practical hostel administration problem. The application supports student registration, room allocation, vacancy checking, record display, and check-out. Classes, objects, methods, loops, conditional statements, arrays, and an ArrayList are used to organize the program and its data. The room allocation logic checks student registration and available capacity before assigning a room. Testing with different inputs helps verify that the system displays suitable messages for successful operations and invalid requests. The basic program is easy to understand and suitable for learning Java, although permanent storage and advanced security would require further development.

7.2 Conclusion

The project provides a simple computerized approach to hostel room allocation and student stay management. It reduces manual record work, helps administrators identify vacant rooms, and supports organized student information. The project also demonstrates the practical use of core Java concepts. Future improvements may include database connectivity, a graphical user interface, login authentication, fee management, attendance tracking, and automated notifications.

8. REFERENCES

The following books and online resources can be used to study Java programming concepts applied in this project:

• Herbert Schildt, Java: The Complete Reference, McGraw Hill. • Oracle, Java Documentation: https://docs.oracle.com/en/java/

• Dev.java, Learn Java: https://dev.java/learn/

• Oracle Java API Documentation — ArrayList: https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/ArrayList.html

• Classroom notes and learning materials on Java programming. — END OF PROJECT REPORT —